

✓ Problem Set 3

Yuqi Gu

✓ 1 Measuring the impact of online word-of-mouth

✓ 1.1 Simple regression

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import statsmodels.api as sm
import statsmodels.formula.api as smf
from scipy import stats
import seaborn as sns
```

```
wdata = pd.read_csv("weibo_data.csv")
```

```
# Display first few rows
print(wdata.head())
```

```
↗
   location  show_id  episode_num  censor_dummy  log_rating  log_tweet \
0  Mainland China    1           1             1    0.475764    0.000000
1  Mainland China    1           2             0    0.468479    0.000000
2  Mainland China    1           3             0    0.581327    1.386294
3  Mainland China    1           4             0    0.547851    0.000000
4  Mainland China    1           5             0    0.483728    1.386294

   av_tweets  day_id  mainland_dummy
0    3.692308     33                1
1    3.692308     34                1
2    3.692308     35                1
3    3.692308     36                1
4    3.692308     37                1
```

wdata

```
↗
   location  show_id  episode_num  censor_dummy  log_rating  log_tweet  av_tweets  day_id  mainland_dummy
0  Mainland China    1           1             1    0.475764    0.000000    3.692308     33                1
1  Mainland China    1           2             0    0.468479    0.000000    3.692308     34                1
2  Mainland China    1           3             0    0.581327    1.386294    3.692308     35                1
3  Mainland China    1           4             0    0.547851    0.000000    3.692308     36                1
4  Mainland China    1           5             0    0.483728    1.386294    3.692308     37                1
...      ...      ...      ...      ...      ...      ...      ...      ...
11422  hongkong    342           33             0    0.476110         NaN    0.000000     51                0
11423  hongkong    342           34             0    0.432756         NaN    0.000000     54                0
11424  hongkong    342           35             0    0.303211         NaN    0.000000     55                0
11425  hongkong    342           36             0    0.436511         NaN    0.000000     56                0
11426  hongkong    342           37             0    0.149712         NaN    0.000000     57                0
```

11427 rows × 9 columns

Next steps: [Generate code with wdata](#) [View recommended plots](#) [New interactive sheet](#)

```
rating_tweets_model = smf.ols(formula = 'log_rating ~ log_tweet', data = wdata)
result = rating_tweets_model.fit()
print(result.summary())
```

```
↗
===== OLS Regression Results =====
Dep. Variable:          log_rating      R-squared:                0.111
Model:                  OLS          Adj. R-squared:           0.111
Method:                 Least Squares   F-statistic:             987.9
Date:                   Sun, 23 Feb 2025   Prob (F-statistic):      2.00e-204
Time:                   05:31:55         Log-Likelihood:          -87.734
No. Observations:      7899             AIC:                    179.5
```

```

Df Residuals:      7897    BIC:      193.4
Df Model:          1
Covariance Type:  nonrobust
=====
              coef    std err          t      P>|t|      [0.025    0.975]
-----
Intercept    0.2664    0.003    81.566    0.000    0.260    0.273
log_tweet    0.0310    0.001    31.431    0.000    0.029    0.033
=====
Omnibus:            2518.070   Durbin-Watson:      0.367
Prob(Omnibus):      0.000   Jarque-Bera (JB):    9432.121
Skew:               1.564   Prob(JB):            0.00
Kurtosis:           7.344   Cond. No.            4.03
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

I don't think this regression gives the causal effect of tweets on show viewership. This is because some popular TV shows with higher ratings might attract more people to watch and so it is common that they will receive more tweets rather than tweets causes the higher viewership for the show. It is a simultaneous causality. Thus, if higher ratings lead to more tweets, we would say the estimated effect of tweets on ratings would be upward biased.

Furthermore, other factors might also influence both the tweets and ratings and they are not included in this regression. This is an omitted variable bias. For example, the shows with popular stars will attract more audience and increase the number of tweets and the ratings of the show. Since this factor is not included in the regression, omitting it might cause upward bias.

Thus, I think the estimate will be biased upwards in general but we also should notice that maybe some shows has drama stories and although their ratings are low, the discussion or the tweets related to them are lot. Thus, it is possible that the biased downwards might exist.

1.2 Geographic Diff-in-diff

▼ (a)

```

wdata_mainland = wdata[wdata['mainland_dummy'] == 1]
model_mainland = smf.ols(formula='log_rating ~C(show_id) + censor_dummy', data=wdata_mainland)
result_mainland = model_mainland.fit()

# Print regression results
print(result_mainland.summary())

```



```

C(show_id) [T.186] -0.4953 0.028 -17.675 0.000 -0.550 -0.440
C(show_id) [T.187] -0.4481 0.027 -16.655 0.000 -0.501 -0.395
C(show_id) [T.188] -0.4413 0.040 -10.968 0.000 -0.520 -0.362
C(show_id) [T.189] -0.5051 0.027 -18.770 0.000 -0.558 -0.452
C(show_id) [T.190] -0.2664 0.047 -5.631 0.000 -0.359 -0.174
C(show_id) [T.191] -0.5113 0.027 -19.004 0.000 -0.564 -0.459
C(show_id) [T.192] -0.3730 0.028 -13.093 0.000 -0.429 -0.317
C(show_id) [T.193] -0.3140 0.027 -11.669 0.000 -0.367 -0.261
censor_dummy -0.0122 0.004 -2.939 0.003 -0.020 -0.004
=====
Omnibus: 925.798 Durbin-Watson: 1.979
Prob(Omnibus): 0.000 Jarque-Bera (JB): 8690.487
Skew: 0.167 Prob(JB): 0.00
Kurtosis: 8.128 Cond. No. 333.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

The coefficient of `censor_dummy` is -0.0122. This means that after accounting for the show-specific factors, during the censorship period, the average `log_rating` for the TV shows of mainland China is decreased by 1.22% compared with the non-censorship period. Also, the p-value is 0.003, which is much smaller than 0.05. Thus, it is statistically significant.

This result is expected since it is expected that Sino Weibo platform engagement will help increase the viewership and improve the ratings, so blocking it will make a negative effect on the `log_rating` and the discussion about the TV shows will decrease.

✓ (b)

```

model_censor = smf.ols(formula='log_rating ~ censor_dummy', data=wdata_mainland)
result_censor = model_censor.fit()

```

```

# Print regression results
print(result_censor.summary())

```

```

=====
OLS Regression Results
=====
Dep. Variable: log_rating R-squared: 0.001
Model: OLS Adj. R-squared: 0.001
Method: Least Squares F-statistic: 6.093
Date: Sun, 23 Feb 2025 Prob (F-statistic): 0.0136
Time: 05:31:56 Log-Likelihood: -550.22
No. Observations: 7899 AIC: 1104.
Df Residuals: 7897 BIC: 1118.
Df Model: 1
Covariance Type: nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	0.3196	0.003	105.685	0.000	0.314	0.326
censor_dummy	0.0286	0.012	2.468	0.014	0.006	0.051

```

=====
Omnibus: 2517.994 Durbin-Watson: 0.268
Prob(Omnibus): 0.000 Jarque-Bera (JB): 7967.313
Skew: 1.639 Prob(JB): 0.00
Kurtosis: 6.670 Cond. No. 3.98
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

It is necessary to control for show fixed effects in the regression since different TV shows have inherent differences like their popularity and quality. If the show fixed effects is not included in the regression, the estimated effect of censor period could be confounded by inherent difference of these shows. Popular shows might get higher ratings regardless of censorship, which leads to biased estimates. Without the show fixed effects, the coefficient of the `censor_dummy` does not capture the true effects of the `censor_dummy` on `log_rating`, but it indicates that other effects on `log_rating` due to unobserved show-specific differences are influencing the estimate.

✓ (c)

```

wdata_hk = wdata[wdata['mainland_dummy'] == 0]
model_hk = smf.ols(formula='log_rating ~ C(show_id) + censor_dummy', data=wdata_hk)
result_hk = model_hk.fit()

print(result_hk.summary())

```



```

C(show_id) [T.287] 0.0032 0.0030 0.0032 0.0031 -0.0035 0.0030
C(show_id) [T.288] 0.4651 0.053 8.696 0.000 0.360 0.570
C(show_id) [T.289] 0.3339 0.053 6.244 0.000 0.229 0.439
C(show_id) [T.290] 0.3621 0.060 6.070 0.000 0.245 0.479
C(show_id) [T.291] 0.5183 0.028 18.468 0.000 0.463 0.573
C(show_id) [T.292] 2.5601 0.028 90.862 0.000 2.505 2.615
C(show_id) [T.294] 2.6095 0.042 61.603 0.000 2.526 2.693
C(show_id) [T.295] 2.9411 0.031 95.702 0.000 2.881 3.001
C(show_id) [T.296] 3.0864 0.031 99.761 0.000 3.026 3.147
C(show_id) [T.298] 2.7465 0.031 89.950 0.000 2.687 2.806
C(show_id) [T.299] 2.5671 0.031 84.076 0.000 2.507 2.627
C(show_id) [T.300] 2.4179 0.031 79.189 0.000 2.358 2.478
C(show_id) [T.304] 2.9807 0.064 46.658 0.000 2.855 3.106
C(show_id) [T.306] 2.9255 0.060 49.045 0.000 2.809 3.042
C(show_id) [T.307] 2.8461 0.064 44.552 0.000 2.721 2.971
C(show_id) [T.308] 3.2094 0.038 83.796 0.000 3.134 3.284
C(show_id) [T.312] 2.7383 0.060 45.907 0.000 2.621 2.855
C(show_id) [T.313] 3.2514 0.038 84.894 0.000 3.176 3.326
C(show_id) [T.314] 2.8646 0.045 64.225 0.000 2.777 2.952
C(show_id) [T.319] 3.2469 0.039 83.305 0.000 3.170 3.323
C(show_id) [T.320] -0.0934 0.037 -2.517 0.012 -0.166 -0.021
C(show_id) [T.321] 0.2315 0.043 5.330 0.000 0.146 0.317
C(show_id) [T.322] 0.6868 0.033 20.832 0.000 0.622 0.751
C(show_id) [T.323] -0.1099 0.041 -2.656 0.008 -0.191 -0.029
C(show_id) [T.324] -0.1339 0.031 -4.297 0.000 -0.195 -0.073
C(show_id) [T.325] 0.4457 0.069 6.424 0.000 0.310 0.582
C(show_id) [T.326] 0.7742 0.053 14.475 0.000 0.669 0.879
C(show_id) [T.327] -0.0339 0.051 -0.663 0.508 -0.134 0.066
C(show_id) [T.328] -0.0094 0.028 -0.335 0.737 -0.064 0.045
C(show_id) [T.329] 0.6008 0.029 21.059 0.000 0.545 0.657
C(show_id) [T.330] 0.1449 0.030 4.859 0.000 0.086 0.203
C(show_id) [T.331] -0.0504 0.053 -0.943 0.346 -0.155 0.054
C(show_id) [T.332] 0.2980 0.024 12.376 0.000 0.251 0.345
C(show_id) [T.333] -0.1323 0.031 -4.306 0.000 -0.193 -0.072
C(show_id) [T.334] -0.1131 0.028 -4.015 0.000 -0.168 -0.058
C(show_id) [T.335] 0.5346 0.053 9.995 0.000 0.430 0.639
C(show_id) [T.336] 0.7099 0.069 10.232 0.000 0.574 0.846
C(show_id) [T.337] 0.1923 0.053 3.595 0.000 0.087 0.297
C(show_id) [T.338] -0.1144 0.053 -2.139 0.033 -0.219 -0.010
C(show_id) [T.339] 0.5488 0.056 9.753 0.000 0.438 0.659
C(show_id) [T.340] -0.0179 0.028 -0.639 0.523 -0.073 0.037
C(show_id) [T.341] -0.0791 0.088 -0.898 0.369 -0.252 0.094
C(show_id) [T.342] 0.2129 0.032 6.683 0.000 0.150 0.275
censor_dummy 0.0106 0.011 0.954 0.340 -0.011 0.032
=====
Omnibus: 611.130 Durbin-Watson: 2.001
Prob(Omnibus): 0.000 Jarque-Bera (JB): 5575.834
Skew: 0.550 Prob(JB): 0.00
Kurtosis: 9.060 Cond. No. 97.1
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

The coefficient of `censor_dummy` is 0.0106. This means that after accounting for the show-specific factors, during the censorship period, the average `log_rating` for the TV shows of Hong Kong is increased by 1.06% compared with non-censorship period. Also, the p-value is 0.34, which is much larger than 0.05. Thus, it is not statistically significant. The censorship dummy is positive but not statistically significant, indicating that Weibo's censorship did not have a meaningful effect on TV show ratings in Hong Kong.

This result is expected since Sino Weibo does not have too much market penetration in Hong Kong and Hong Kong residents mainly use Twitter. Thus, the censorship of Weibo does not influence a lot for the ratings in Hong Kong since the usage of Weibo is not a lot.

✓ (d)

```

d_model = smf.ols(formula='log_rating ~ C(show_id) + mainland_dummy + censor_dummy + mainland_dummy:censor_dummy', data=wda)
d_result = d_model.fit()

print(d_result.summary())

```



```

C(show_id) [T.308]      2.6083      0.024      107.260      0.000      2.561      2.656
C(show_id) [T.312]      2.1372      0.042      50.976      0.000      2.055      2.219
C(show_id) [T.313]      2.6503      0.024      108.988      0.000      2.603      2.698
C(show_id) [T.314]      2.2635      0.030      76.181      0.000      2.205      2.322
C(show_id) [T.319]      2.6458      0.025      106.216      0.000      2.597      2.695
C(show_id) [T.320]      -0.6945      0.023      -29.872      0.000      -0.740      -0.649
C(show_id) [T.321]      -0.3696      0.029      -12.872      0.000      -0.426      -0.313
C(show_id) [T.322]      0.0857      0.019      4.400      0.000      0.048      0.124
C(show_id) [T.323]      -0.7110      0.027      -26.345      0.000      -0.764      -0.658
C(show_id) [T.324]      -0.7350      0.018      -41.443      0.000      -0.770      -0.700
C(show_id) [T.325]      -0.1554      0.050      -3.134      0.002      -0.253      -0.058
C(show_id) [T.326]      0.1731      0.037      4.679      0.000      0.101      0.246
C(show_id) [T.327]      -0.6350      0.035      -18.086      0.000      -0.704      -0.566
C(show_id) [T.328]      -0.6105      0.014      -42.239      0.000      -0.639      -0.582
C(show_id) [T.329]      -0.0003      0.015      -0.022      0.982      -0.030      0.029
C(show_id) [T.330]      -0.4562      0.016      -27.820      0.000      -0.488      -0.424
C(show_id) [T.331]      -0.6515      0.037      -17.610      0.000      -0.724      -0.579
C(show_id) [T.332]      -0.3031      0.010      -30.894      0.000      -0.322      -0.284
C(show_id) [T.333]      -0.7334      0.017      -42.349      0.000      -0.767      -0.699
C(show_id) [T.334]      -0.7142      0.015      -48.633      0.000      -0.743      -0.685
C(show_id) [T.335]      -0.0665      0.037      -1.797      0.072      -0.139      0.006
C(show_id) [T.336]      0.1088      0.050      2.195      0.028      0.012      0.206
C(show_id) [T.337]      -0.4088      0.037      -11.049      0.000      -0.481      -0.336
C(show_id) [T.338]      -0.7155      0.037      -19.339      0.000      -0.788      -0.643
C(show_id) [T.339]      -0.0523      0.039      -1.334      0.182      -0.129      0.025
C(show_id) [T.340]      -0.6190      0.014      -42.826      0.000      -0.647      -0.591
C(show_id) [T.341]      -0.6801      0.064      -10.629      0.000      -0.806      -0.555
C(show_id) [T.342]      -0.3882      0.018      -21.073      0.000      -0.424      -0.352
mainland_dummy          -0.2226      0.030      -7.488      0.000      -0.281      -0.164
censor_dummy            0.0106      0.008      1.267      0.205      -0.006      0.027
mainland_dummy:censor_dummy -0.0227      0.010      -2.328      0.020      -0.042      -0.004
=====
Omnibus:                2111.802      Durbin-Watson:          1.991
Prob(Omnibus):          0.000      Jarque-Bera (JB):       31172.319
Skew:                   0.455      Prob(JB):               0.00
Kurtosis:               11.040      Cond. No.               2.17e+15
=====

```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The smallest eigenvalue is 3.79e-27. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

The coefficient of mainland_dummy is -0.2226 and the p-value is 0.000, which is much smaller than 0.05. It shows that, after accounting for show fixed effects, on average, the ratings for the shows in Mainland China are 0.2226 lower compared to those in Hong Kong, before the censorship event.

The coefficient of censor_dummy is 0.0106 and the p-value is 0.205, which is larger than 0.05. It indicates the effect of censorship across both of the Mainland China and Hong Kong. The coefficient is positive but not statistically significant and it is just 0.0106. It suggests that the censorship did not significantly impact the ratings of the TV shows for both Mainland China and Hong Kong.

The coefficient of mainland_dummy:censor_dummy is -0.0227 and the p-value is 0.020, which is smaller than 0.05. This is the difference-in-difference estimate. It measures the difference between how much the censorship affected Mainland China relative to Hong Kong. The negative and significant coefficient indicates that censorship led to a decline in ratings in Mainland China relative to Hong Kong. To be specific, the censorship cause the ratings of TV shows in Mainland China drop by 0.0227 compared with that in Hong Kong.

✓ 1.3 Across-show Diff-in-diff

✓ (a)

```

wdata_mainland['low_tweet'] = (wdata_mainland['av_tweets'] < 5)
wdata_mainland['medium_tweet'] = ((wdata_mainland['av_tweets'] >= 5) & (wdata_mainland['av_tweets'] < 100))
wdata_mainland['high_tweet'] = (wdata_mainland['av_tweets'] >= 100)

```

 <ipython-input-11-f2a3bdebc553>:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
wdata_mainland['low_tweet'] = (wdata_mainland['av_tweets'] < 5)

<ipython-input-11-f2a3bdebc553>:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
wdata_mainland['medium_tweet'] = ((wdata_mainland['av_tweets'] >= 5) & (wdata_mainland['av_tweets'] < 100))

<ipython-input-11-f2a3bdebc553>:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: [https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-wdata_mainland\['high_tweet'\] = \(wdata_mainland\['av_tweets'\] >= 100\)](https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-wdata_mainland['high_tweet'] = (wdata_mainland['av_tweets'] >= 100))

```
ltweet_model = smf.ols(formula='log_rating ~ C(show_id) + censor_dummy', data=wdata_mainland[wdata_mainland['low_tweet'] ==
mtweet_model = smf.ols(formula='log_rating ~ C(show_id) + censor_dummy', data=wdata_mainland[wdata_mainland['medium_tweet']
htweet_model = smf.ols(formula='log_rating ~ C(show_id) + censor_dummy', data=wdata_mainland[wdata_mainland['high_tweet'] ==

ltweet_result = ltweet_model.fit()
print(ltweet_result.summary())
```

```

C(show_id) [T.82]      -0.4150    0.022   -19.236    0.000   -0.457   -0.373
C(show_id) [T.85]      -0.3605    0.028   -12.854    0.000   -0.415   -0.306
C(show_id) [T.86]      -0.4327    0.023   -18.867    0.000   -0.478   -0.388
C(show_id) [T.89]      -0.4497    0.022   -20.841    0.000   -0.492   -0.407
C(show_id) [T.90]      -0.4113    0.022   -19.062    0.000   -0.454   -0.369
C(show_id) [T.92]      -0.1811    0.021    -8.813    0.000   -0.221   -0.141
C(show_id) [T.93]      -0.3934    0.021   -18.540    0.000   -0.435   -0.352
C(show_id) [T.100]     -0.4239    0.022   -19.646    0.000   -0.466   -0.382
C(show_id) [T.102]     -0.1690    0.029    -5.760    0.000   -0.227   -0.111
C(show_id) [T.105]     -0.4836    0.022   -21.866    0.000   -0.527   -0.440
C(show_id) [T.106]     -0.4860    0.028   -17.329    0.000   -0.541   -0.431
C(show_id) [T.107]     -0.1580    0.036    -4.446    0.000   -0.228   -0.088
C(show_id) [T.108]     -0.3897    0.038   -10.262    0.000   -0.464   -0.315
C(show_id) [T.110]     -0.4937    0.022   -22.881    0.000   -0.536   -0.451
C(show_id) [T.113]     -0.4475    0.022   -20.739    0.000   -0.490   -0.405
C(show_id) [T.115]     -0.0938    0.022    -4.302    0.000   -0.136   -0.051
C(show_id) [T.117]     -0.5071    0.022   -23.502    0.000   -0.549   -0.465
C(show_id) [T.118]     -0.2397    0.024   -10.118    0.000   -0.286   -0.193
C(show_id) [T.123]     -0.4043    0.022   -18.736    0.000   -0.447   -0.362
C(show_id) [T.124]     -0.1182    0.022    -5.383    0.000   -0.161   -0.075
C(show_id) [T.125]     -0.4518    0.022   -20.941    0.000   -0.494   -0.410
C(show_id) [T.131]     -0.3841    0.022   -17.803    0.000   -0.426   -0.342
C(show_id) [T.132]     -0.4152    0.021   -20.212    0.000   -0.456   -0.375
C(show_id) [T.139]      0.0177    0.038     0.466    0.641   -0.057     0.092
C(show_id) [T.140]     -0.0783    0.031    -2.518    0.012   -0.139   -0.017
C(show_id) [T.141]     -0.2207    0.036    -6.211    0.000   -0.290   -0.151
C(show_id) [T.144]     -0.4396    0.023   -19.379    0.000   -0.484   -0.395
C(show_id) [T.146]     -0.3384    0.022   -15.686    0.000   -0.381   -0.296
C(show_id) [T.149]     -0.4074    0.025   -16.506    0.000   -0.456   -0.359
C(show_id) [T.152]     -0.3332    0.021   -15.878    0.000   -0.374   -0.292
C(show_id) [T.154]     -0.0784    0.022    -3.634    0.000   -0.121   -0.036
C(show_id) [T.156]     -0.4342    0.032   -13.454    0.000   -0.497   -0.371
C(show_id) [T.157]     -0.0140    0.022    -0.651    0.515   -0.056     0.028
C(show_id) [T.160]     -0.2388    0.024   -10.077    0.000   -0.285   -0.192
C(show_id) [T.161]     -0.5116    0.025   -20.555    0.000   -0.560   -0.463
C(show_id) [T.163]     -0.3379    0.038    -8.906    0.000   -0.412   -0.263
C(show_id) [T.164]     -0.4672    0.034   -13.860    0.000   -0.533   -0.401
C(show_id) [T.165]     -0.5188    0.022   -23.665    0.000   -0.562   -0.476
C(show_id) [T.168]     -0.3328    0.027   -12.300    0.000   -0.386   -0.280
C(show_id) [T.169]     -0.5156    0.022   -23.898    0.000   -0.558   -0.473
C(show_id) [T.171]     -0.4047    0.022   -18.729    0.000   -0.447   -0.362
C(show_id) [T.178]     -0.5084    0.022   -23.563    0.000   -0.551   -0.466
C(show_id) [T.180]     -0.4568    0.023   -20.203    0.000   -0.501   -0.412
C(show_id) [T.181]     -0.3678    0.030   -12.202    0.000   -0.427   -0.309
C(show_id) [T.186]     -0.4950    0.022   -22.030    0.000   -0.539   -0.451
C(show_id) [T.189]     -0.5049    0.022   -23.403    0.000   -0.547   -0.463
C(show_id) [T.190]     -0.2670    0.038    -7.039    0.000   -0.341   -0.193
C(show_id) [T.191]     -0.5112    0.022   -23.694    0.000   -0.554   -0.469
censor_dummy          -0.0069    0.005    -1.326    0.185   -0.017     0.003
=====
Omnibus:                500.093   Durbin-Watson:                1.664
Prob(Omnibus):           0.000   Jarque-Bera (JB):             3305.218
Skew:                    0.519   Prob(JB):                     0.00
Kurtosis:                7.714   Cond. No.                     149.
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
mtweet_result = mtweet_model.fit()
print(mtweet_result.summary())
```



```

C(show_id) [T.95]      -0.0130      0.019      -30.211      0.000      -0.111      -0.030
C(show_id) [T.98]      -0.4227      0.035      -12.008      0.000      -0.492      -0.354
C(show_id) [T.99]      -0.3720      0.032      -11.573      0.000      -0.435      -0.309
C(show_id) [T.104]     -0.7126      0.018      -39.107      0.000      -0.748      -0.677
C(show_id) [T.109]     -0.7833      0.019      -42.232      0.000      -0.820      -0.747
C(show_id) [T.114]     -0.7752      0.025      -30.545      0.000      -0.825      -0.725
C(show_id) [T.116]     -0.3477      0.019      -17.948      0.000      -0.386      -0.310
C(show_id) [T.120]     -0.8354      0.019      -44.897      0.000      -0.872      -0.799
C(show_id) [T.121]     -0.7348      0.017      -44.008      0.000      -0.768      -0.702
C(show_id) [T.126]     -0.6321      0.024      -26.140      0.000      -0.679      -0.585
C(show_id) [T.127]     -0.7708      0.022      -35.312      0.000      -0.814      -0.728
C(show_id) [T.128]     -0.1119      0.023      -4.870      0.000      -0.157      -0.067
C(show_id) [T.134]     -0.7158      0.019      -38.470      0.000      -0.752      -0.679
C(show_id) [T.135]     -0.8341      0.024      -34.479      0.000      -0.882      -0.787
C(show_id) [T.136]     -0.6189      0.034      -18.455      0.000      -0.685      -0.553
C(show_id) [T.142]     -0.3756      0.021      -17.782      0.000      -0.417      -0.334
C(show_id) [T.145]     -0.6131      0.021      -28.560      0.000      -0.655      -0.571
C(show_id) [T.147]     -0.4359      0.022      -19.773      0.000      -0.479      -0.393
C(show_id) [T.151]     -0.6712      0.021      -32.022      0.000      -0.712      -0.630
C(show_id) [T.153]     -0.5515      0.031      -17.811      0.000      -0.612      -0.491
C(show_id) [T.155]     -0.6044      0.017      -36.089      0.000      -0.637      -0.572
C(show_id) [T.158]     -0.8310      0.019      -42.669      0.000      -0.869      -0.793
C(show_id) [T.159]     -0.6353      0.021      -30.286      0.000      -0.676      -0.594
C(show_id) [T.166]     -0.1449      0.019      -7.786      0.000      -0.181      -0.108
C(show_id) [T.167]     -0.5527      0.034      -16.481      0.000      -0.618      -0.487
C(show_id) [T.170]     -0.7661      0.022      -34.097      0.000      -0.810      -0.722
C(show_id) [T.174]     -0.8447      0.019      -45.396      0.000      -0.881      -0.808
C(show_id) [T.175]     -0.6796      0.028      -24.050      0.000      -0.735      -0.624
C(show_id) [T.176]     -0.8133      0.019      -43.710      0.000      -0.850      -0.777
C(show_id) [T.177]     -0.7649      0.048      -16.065      0.000      -0.858      -0.672
C(show_id) [T.182]     -0.8184      0.018      -45.459      0.000      -0.854      -0.783
C(show_id) [T.184]     -0.8141      0.029      -28.033      0.000      -0.871      -0.757
C(show_id) [T.185]     -0.8132      0.030      -27.160      0.000      -0.872      -0.754
C(show_id) [T.187]     -0.7766      0.019      -41.740      0.000      -0.813      -0.740
C(show_id) [T.188]     -0.7704      0.035      -21.889      0.000      -0.839      -0.701
C(show_id) [T.193]     -0.6425      0.019      -34.529      0.000      -0.679      -0.606
censor_dummy          -0.0042      0.007      -0.610      0.542      -0.018      0.009
=====
Omnibus:                344.985      Durbin-Watson:          2.063
Prob(Omnibus):           0.000      Jarque-Bera (JB):       1518.722
Skew:                    0.493      Prob(JB):               0.00
Kurtosis:                6.377      Cond. No.               71.0
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

htweet_result = htweet_model.fit()
print(htweet_result.summary())

```

```

Df Residuals:      1506      BIC:      -1866.
Df Model:           42
Covariance Type:    nonrobust
=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept      0.7549      0.016      48.152      0.000      0.724      0.786
C(show_id) [T.5]  0.4143      0.022      18.774      0.000      0.371      0.458
C(show_id) [T.6] -0.1895      0.025      -7.650      0.000     -0.238     -0.141
C(show_id) [T.7] -0.3019      0.022     -13.679      0.000     -0.345     -0.259
C(show_id) [T.9] -0.3437      0.026     -13.311      0.000     -0.394     -0.293
C(show_id) [T.13] -0.5975      0.033     -18.304      0.000     -0.662     -0.533
C(show_id) [T.25] -0.0710      0.028      -2.493      0.013     -0.127     -0.015
C(show_id) [T.36] -0.4552      0.043     -10.492      0.000     -0.540     -0.370
C(show_id) [T.38] -0.2068      0.022      -9.373      0.000     -0.250     -0.164
C(show_id) [T.39] -0.6109      0.024     -25.190      0.000     -0.658     -0.563
C(show_id) [T.44] -0.3819      0.043      -8.803      0.000     -0.467     -0.297
C(show_id) [T.45] -0.5401      0.018     -29.593      0.000     -0.576     -0.504
C(show_id) [T.56] -0.3873      0.022     -17.481      0.000     -0.431     -0.344
C(show_id) [T.63] -0.3339      0.019     -17.451      0.000     -0.371     -0.296
C(show_id) [T.80]  0.3850      0.043       8.873      0.000      0.300      0.470
C(show_id) [T.83] -0.1471      0.025      -5.847      0.000     -0.196     -0.098
C(show_id) [T.87] -0.0328      0.027      -1.220      0.223     -0.085      0.020
C(show_id) [T.88] -0.2633      0.028      -9.473      0.000     -0.318     -0.209
C(show_id) [T.91]  0.2441      0.026       9.364      0.000      0.193      0.295
C(show_id) [T.94] -0.3980      0.036     -11.049      0.000     -0.469     -0.327
C(show_id) [T.96] -0.2230      0.029      -7.796      0.000     -0.279     -0.167
C(show_id) [T.97] -0.6572      0.031     -21.359      0.000     -0.718     -0.597
C(show_id) [T.101] -0.3653      0.027     -13.309      0.000     -0.419     -0.311
C(show_id) [T.103]  0.0285      0.027       1.049      0.295     -0.025      0.082
C(show_id) [T.111] -0.2754      0.043      -6.347      0.000     -0.360     -0.190
C(show_id) [T.112] -0.4045      0.036     -11.226      0.000     -0.475     -0.334

```

```

C(show_id) [T.148] -0.0041 0.031 -10.210 0.000 -0.011 -0.551
C(show_id) [T.150] -0.5709 0.022 -25.975 0.000 -0.614 -0.528
C(show_id) [T.162] -0.0875 0.028 -3.143 0.002 -0.142 -0.033
C(show_id) [T.172] -0.6254 0.023 -26.910 0.000 -0.671 -0.580
C(show_id) [T.173] -0.3985 0.028 -14.314 0.000 -0.453 -0.344
C(show_id) [T.179] -0.6133 0.030 -20.272 0.000 -0.673 -0.554
C(show_id) [T.183] -0.7044 0.038 -18.343 0.000 -0.780 -0.629
C(show_id) [T.192] -0.5668 0.025 -22.314 0.000 -0.617 -0.517
censor_dummy -0.0335 0.011 -2.930 0.003 -0.056 -0.011
=====
Omnibus: 168.640 Durbin-Watson: 2.146
Prob(Omnibus): 0.000 Jarque-Bera (JB): 1059.925
Skew: -0.283 Prob(JB): 6.92e-231
Kurtosis: 7.013 Cond. No. 34.8
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

For low-tweet shows, which is < 5 tweets per episode, we find that the coefficient of `censor_dummy` is -0.0069 (which is close to 0) and its p-value is 0.185. It indicates that there is no significant effect of censorship for low-tweet shows. After accounting for the show fixed effects, the average `log_rating` of low-tweet shows decrease by 0.0069 during the censorship compared with non-censorship period. This makes sense because these shows were not heavily discussed on Weibo, we can know that the censorship did not affect their ratings too much.

For medium-tweet shows, which is 5-100 tweets per episode, we find that the coefficient of `censor_dummy` is -0.0042 and the p-value is 0.542. Similar to low-tweet shows, this coefficient indicates that there is no significant effect of censorship for medium-tweet shows. After accounting for the show fixed effects, the average `log_rating` of medium-tweet shows decrease by 0.0042 during the censorship compared with non-censorship period. This makes sense since these shows had some Weibo engagement, but their viewership was likely not highly dependent on Weibo.

For high-tweet shows, which is ≥ 100 tweets per episode, we find that the coefficient of `censor_dummy` is -0.0335 and p-value is 0.003. Thus, there is significant negative impact of censorship on high-tweet shows. It indicates that after accounting for the show fixed effects, the average `log_rating` of high-tweet shows decrease by 0.0335 during the censorship compared with non-censorship period. This suggests that highly-discussed shows experienced a drop in ratings due to the lack of Weibo engagement during the censorship period. This supports that Weibo online discussions contribute to viewership for highly popular shows.

In conclusion, censorship had almost no impact on shows that weren't discussed a lot on Weibo. Censorship significantly reduced ratings for highly-discussed shows, suggesting that online Weibo discussions have an important role in ratings of TV shows.

✓ (b)

```
did_model = smf.ols(formula='log_rating ~ C(show_id) + censor_dummy + medium_tweet*censor_dummy + high_tweet*censor_dummy',
```

```
did_result = did_model.fit()
print(did_result.summary())
```




```

C(show_id) [T.187] -0.2141 0.012 -18.336 0.000 -0.237 -0.191
C(show_id) [T.188] -0.2079 0.032 -6.566 0.000 -0.270 -0.146
C(show_id) [T.189] -0.5049 0.027 -18.773 0.000 -0.558 -0.452
C(show_id) [T.190] -0.2670 0.047 -5.647 0.000 -0.360 -0.174
C(show_id) [T.191] -0.5112 0.027 -19.007 0.000 -0.564 -0.458
C(show_id) [T.192] -0.2886 0.015 -19.394 0.000 -0.318 -0.259
C(show_id) [T.193] -0.0800 0.012 -6.848 0.000 -0.103 -0.057
medium_tweet[T.True] -0.2340 0.024 -9.750 0.000 -0.281 -0.187
high_tweet[T.True] -0.0806 0.024 -3.373 0.001 -0.127 -0.034
censor_dummy -0.0069 0.006 -1.063 0.288 -0.020 0.006
medium_tweet[T.True]:censor_dummy 0.0027 0.009 0.282 0.778 -0.016 0.021
high_tweet[T.True]:censor_dummy -0.0266 0.011 -2.479 0.013 -0.048 -0.006
=====
Omnibus: 925.888 Durbin-Watson: 1.982
Prob(Omnibus): 0.000 Jarque-Bera (JB): 8692.408
Skew: 0.167 Prob(JB): 0.00
Kurtosis: 8.128 Cond. No. 2.66e+15
=====

```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The smallest eigenvalue is 1.37e-27. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

The intercept is 0.5573. The expected rating value when there is no censorship and for low tweet shows is 0.5573 after accounting for the fixed effects of shows.

The coefficient of `censor_dummy` is -0.0069 and its p-value is 0.288. It represents that the average rating of low-tweet shows during censorship is 0.0069 lower than the average rating of low-tweet shows during non-censorship period. And the p-value is larger than 0.05, so it is not statistically significant. The effect of censorship on low tweet shows is -0.0069. Thus, the expected rating value when there is censorship and for low tweet shows is $0.5573 - 0.0069 = 0.5504$ after accounting for the fixed effects of shows. Thus, the censorship causes the rating for low tweet show becomes a little bit lower, which is from 0.5573 to 0.5504 and it is not significant.

The coefficient of `medium_tweet[T.True]` is -0.2340 and its p-value is 0.000, which is much smaller than 0.05. The expected rating value when there is no censorship and for medium tweet shows is $0.5573 - 0.234 = 0.3233$ after accounting for the fixed effects of shows. This coefficient represents the difference between the average rating of medium tweet shows and that of low tweet shows when there is no censorship. It is significantly important.

The coefficient of `medium_tweet[T.True]:censor_dummy` is 0.0027 and its p-value is 0.778. It is the difference in censorship effect for the rating of medium tweet shows relative to the ratings of low tweet shows. It represents that the positive effect that censorship brings to medium tweet shows is larger than the effect that censorship brings to low tweet shows. Specifically, the positive effect of censorship on rating of medium tweet shows is 0.0027 compared with the effect of censorship on rating of low tweet shows. And the p-value is larger than 0.05, so it is not statistically significant. The expected rating value when there is censorship and for medium tweet show is $0.5573 - 0.0069 - 0.234 + 0.0027 = 0.3191$ after accounting for the fixed effects of shows. Thus, the censorship causes the rating for medium tweet show becomes a little bit lower, which is from 0.3233 to 0.3191 and it is not significant.

The coefficient of `high_tweet[T.True]` is -0.0806 and its p-value is 0.001, which is smaller than 0.05. The expected rating value when there is no censorship and for high tweet shows is $0.5573 - 0.0806 = 0.4767$ after accounting for the fixed effects of shows. This coefficient represents the difference between the average rating of high tweet shows and that of low tweet shows when there is no censorship. It is significantly important.

The coefficient of `high_tweet[T.True]:censor_dummy` is -0.0266 and its p-value is 0.013. It is the difference in censorship effect for the rating of high tweet shows relative to the ratings of low tweet shows. It represents that the negative effect that censorship brings to high-tweet shows is larger than the effect that censorship brings to low-tweet shows. Specifically, the negative effect of censorship on rating of high-tweet shows is 0.0266 compared with the effect of censorship on rating of low-tweet shows. And the p-value is smaller than 0.05, so it is statistically significant. The expected rating value when there is censorship and for high tweet show is $0.5573 - 0.0069 - 0.0806 - 0.0266 = 0.4432$ after accounting for the fixed effects of shows. Thus, the censorship causes the rating for high tweet show becomes lower, which is from 0.4767 to 0.4432 and it is statistically significant.

Censorship mostly affected highly discussed shows in Mainland China, reinforcing the idea that online discussion influences TV viewership and ratings. Low and medium engagement shows were not significantly impacted.

✓ (c)

Based on geographic difference-in-difference approach, we find that the censorship event had a significant negative impact on ratings in Mainland China relative to Hong Kong. This suggests that Weibo engagement played a role in driving viewership in China, while Hong Kong audiences were less dependent on Weibo.

Based on the across-show difference-in-difference approach, we find that the effect of censorship on ratings of TV shows in Mainland China is based on the activity levels. For high tweet shows, the censorship will cause significant drop in ratings. On the contrary, there is no significant impact of censorship on low or medium tweet shows. The effect of censorship was strongest for high tweet shows, indicating that

Weibo discussions brings viewership and has effects on ratings for popular shows. Shows with little to moderate online discussion were almost unaffected.

Geographic diff-in-diff tells us about the effect of censorship on an entire market (Mainland China vs. Hong Kong), but it does not differentiate between shows that relied on Weibo more or less. Across-show diff-in-diff directly measures how censorship affected shows with different levels of Weibo engagement. It tells us not only just whether censorship had an effect, but who was affected the most (high-tweet shows).

Thus, the across-show diff-in-diff is more informative regarding the impact of the censorship on ratings since it not only shows the censorship brings negative effects on ratings for the shows in mainland China since people in mainland China mainly use Sino Weibo for discussions, but also helps identify what kind of shows with what activity levels will be affected the most by censorship. And the geographic diff-in-diff only gives us the effect of censorship on the ratings of shows in mainland China compared with Hong Kong where people do not use Weibo a lot.